

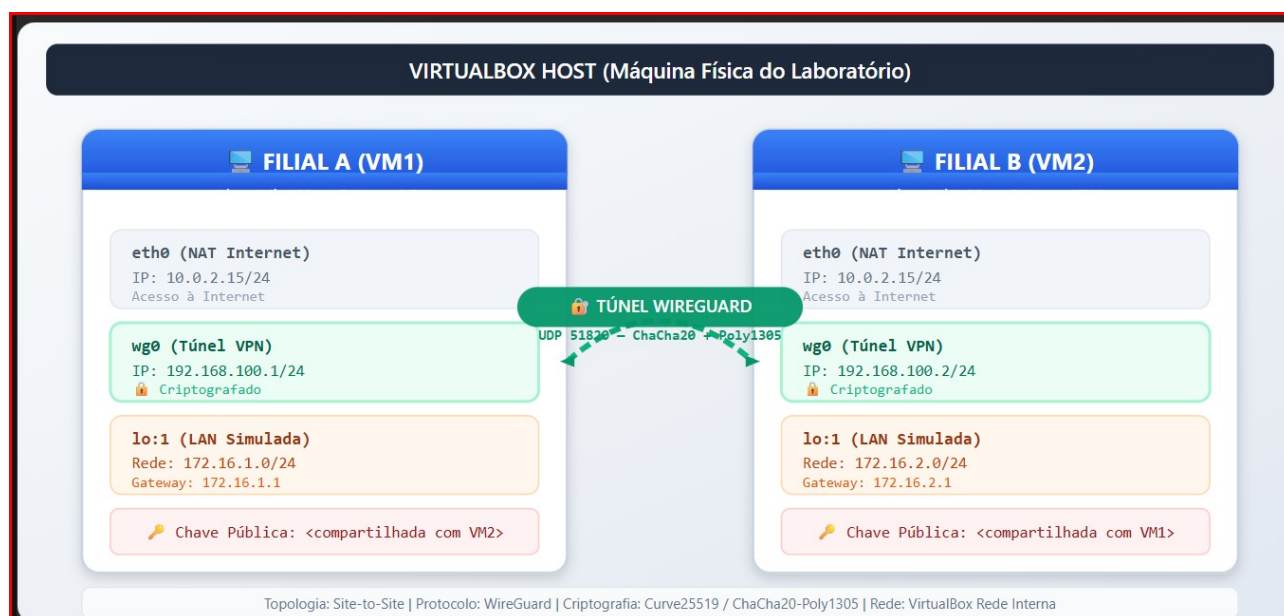
ISG012-04 - Redes Privadas Virtuais (VPN) – Prática Lab

1. Objetivo do Laboratório

Implementar uma VPN site-to-site simples utilizando **WireGuard** entre duas máquinas virtuais **Linux Mint 22**, interconectando dois segmentos de rede distintos (simulando filiais). Ao final, o aluno será capaz de:

- Gerar pares de chaves públicas/privadas Curve25519
- Configurar interfaces WireGuard (**wg0**) em modo site-to-site
- Estabelecer túnel criptografado entre gateways VPN
- Testar conectividade segura entre redes privadas
- Diagnosticar problemas comuns de conectividade

2. Topologia do Laboratório



Nota: Como ambas as VMs usam NAT padrão do VirtualBox (10.0.2.0/24), simulamos redes LAN distintas via alias de loopback (lo:1). Isso evita configuração complexa de redes internas no VirtualBox.

3. Preparação do Ambiente VirtualBox

3.1 Criar as VMs

1. **VM1 – Filial A:** Linux Mint 22, 2GB RAM, 1 CPU, disco 20GB
2. **VM2 – Filial B:** Linux Mint 22, 2GB RAM, 1 CPU, disco 20GB

3.2 Configuração de Rede

- Em ambas as VMs, configure o **Adapter 1** como **NAT** (padrão do VirtualBox)
- Isso permite acesso à Internet para instalação de pacotes

- As VMs estarão na mesma rede NAT 10.0.2.0/24 (simulação da Internet pública)

3.3 Ajuste de Port Forwarding (para conectividade VM → VM)

Como ambas usam NAT, o VirtualBox isola o tráfego entre elas. Habilite **Port Forwarding** no Adapter 1 de cada VM:

Na VM1 (Filial A):

- Nome: `wg-filial-a`
- Protocolo: UDP
- Host IP: `127.0.0.1`
- Host Port: `51820`
- Guest IP: `10.0.2.15`
- Guest Port: `51820`

Na VM2 (Filial B):

- Nome: `wg-filial-b`
- Protocolo: UDP
- Host IP: `127.0.0.1`
- Host Port: `51821`
- Guest IP: `10.0.2.15`
- Guest Port: `51820`

Alternativa mais simples: configure o **Adapter 2** em ambas as VMs como **Rede Interna** (nome: `vpn-lab`). Isso elimina a necessidade de port forwarding. Este roteiro usará essa abordagem (mais didática).

3.4 Configuração Recomendada (Rede Interna)

VM	Adapter 1	Adapter 2
VM1	NAT (Internet)	Rede Interna: <code>vpn-lab</code>
VM2	NAT (Internet)	Rede Interna: <code>vpn-lab</code>

Após configurar, anote os IPs do Adapter 2 (rede interna):

```
bash
# Em ambas as VMs, após boot
ip addr show
```

Tipicamente serão atribuídos IPs manualmente ou via DHCP interno. **Recomenda-se IP estático:**

- VM1 (Adapter 2): `192.168.56.101/24`
- VM2 (Adapter 2): `192.168.56.102/24`

4. Instalação do WireGuard

4.1 Em ambas as VMs (Filial A e Filial B)

```
bash
sudo apt update
sudo apt install -y wireguard wireguard-tools
```

Verifique a instalação:

```
bash
wg --version
```

5. Geração de Chaves

5.1 Filial A (VM1)

```
bash
# Criar diretório de configuração
sudo mkdir -p /etc/wireguard
sudo chmod 700 /etc/wireguard

# Gerar chaves privada e pública
cd /etc/wireguard
sudo wg genkey | sudo tee privatekey | sudo wg pubkey | sudo tee publickey

# Verificar (anote os valores!)
sudo cat privatekey    # Ex: aBcD... (mantenha em SEGREDO)
sudo cat publickey     # Ex: xYzW... (será compartilhada)
```

5.2 Filial B (VM2)

```
bash
sudo mkdir -p /etc/wireguard
sudo chmod 700 /etc/wireguard
cd /etc/wireguard
sudo wg genkey | sudo tee privatekey | sudo wg pubkey | sudo tee publickey
sudo cat privatekey
sudo cat publickey
```

Importante: Em implementações de produção, a chave privada NUNCA deve sair da máquina. Permissões 0400 são obrigatórias.

6. Simulação de Redes LAN Locais (Loopback Alias)

Para simular redes locais distintas sem complicar a topologia do VirtualBox:

6.1 Filial A – Rede 172.16.1.0/24

```
bash
sudo ip addr add 172.16.1.1/24 dev lo label lo:1
# Verificar
ip addr show lo
ping -c 2 172.16.1.1
```

6.2 Filial B – Rede 172.16.2.0/24

```
bash
sudo ip addr add 172.16.2.1/24 dev lo label lo:1
# Verificar
ip addr show lo
ping -c 2 172.16.2.1
```

7. Configuração WireGuard Site-to-Site

7.1 Filial A (VM1) – /etc/wireguard/wg0.conf

```
bash
sudo nano /etc/wireguard/wg0.conf
```

Conteúdo:

```
[Interface]
PrivateKey = <CHAVE_PRIVADA_VM1>
Address = 192.168.100.1/24
ListenPort = 51820

[Peer]
PublicKey = <CHAVE_PUBLICA_VM2>
AllowedIPs = 192.168.100.2/32, 172.16.2.0/24
Endpoint = 192.168.56.102:51820
PersistentKeepalive = 25
```

7.2 Filial B (VM2) – /etc/wireguard/wg0.conf

```
bash
sudo nano /etc/wireguard/wg0.conf
```

Conteúdo:

```
[Interface]
PrivateKey = <CHAVE_PRIVADA_VM2>
Address = 192.168.100.2/24
ListenPort = 51820
```

[Peer]

```
PublicKey = <CHAVE_PUBLICA_VM1>  
AllowedIPs = 192.168.100.1/32, 172.16.1.0/24  
Endpoint = 192.168.56.101:51820  
PersistentKeepalive = 25
```

Explicação didática dos campos:

- Address: IP virtual dentro do túnel
- VPN - AllowedIPs: Redes acessíveis através deste peer (roteamento)
- Endpoint: IP público/visível do peer remoto + porta UDP
- PersistentKeepalive: Mantém NAT/firewall aberto (pacotes a cada 25s)

8. Ativação e Testes

8.1 Ativar interfaces em ambas as Vms

```
bash  
# Filial A e Filial B  
sudo wg-quick up wg0  
  
# Verificar status  
sudo wg show
```

Saída esperada (exemplo VM1):

```
interface: wg0  
  public key: <chave pública VM1>  
  private key: (hidden)  
  listening port: 51820  
  
peer: <chave pública VM2>  
  endpoint: 192.168.56.102:51820  
  allowed ips: 192.168.100.2/32, 172.16.2.0/24  
  latest handshake: 5 seconds ago  
  transfer: 1.21 KiB received, 1.45 KiB sent  
  persistent keepalive: every 25 seconds
```

8.2 Teste 1 – Ping pelo túnel VPN

```
bash  
# Da VM1, testar conectividade com VM2 pelo IP do túnel  
ping -c 4 192.168.100.2  
  
# Da VM2, testar VM1  
ping -c 4 192.168.100.1
```

8.3 Teste 2 – Acesso à rede LAN simulada do peer

```
vash
# Da VM1 (Filial A), acessar rede LAN da VM2 (Filial B)
ping -c 4 172.16.2.1

# Da VM2 (Filial B), acessar rede LAN da VM1 (Filial A)
ping -c 4 172.16.1.1
```

Resultado esperado: Todos os pings devem responder com sucesso, provando que o túnel criptografado está transportando tráfego entre as redes privadas simuladas.

8.4 Teste 3 – Captura de pacotes (prova de criptografia)

Em uma terceira janela/terminal (ou em uma das VMs, na interface de rede física):

```
# Instalar tcpdump se necessário
sudo apt install -y tcpdump

# Capturar tráfego na interface física (não no wg0!)
sudo tcpdump -i enp0s8 -n udp port 51820 -X
```

Execute um ping pelo túnel em outro terminal. Você verá pacotes UDP na porta 51820, mas **os dados do ping estarão criptografados** (não legíveis no tcpdump da interface física).

Para comparar, capture na interface wg0:

```
nash
sudo tcpdump -i wg0 -n icmp
```

Aqui você verá os pacotes ICMP em texto claro (dentro do túnel já descriptografado).

9. Verificação de Segurança

9.1 Permissões das chaves

```
bash
sudo ls -la /etc/wireguard/
# Esperado: privatekey com permissões 0400 (ou 0600)
```

9.2 Roteamento

```
bash
ip route show
# Deve aparecer rotas para 172.16.x.0/24 via wg0
```

9.3 Firewall (opcional, para reforço didático)

```
bash
```

```
# Permitir apenas WireGuard na interface física
sudo iptables -A INPUT -i enp0s8 -p udp --dport 51820 -j ACCEPT
sudo iptables -A INPUT -i enp0s8 -j DROP
```

10. Troubleshooting Rápido

Problema	Causa Provável	Solução
wg show não mostra handshake	Firewall bloqueando UDP 51820	sudo ufw allow 51820/udp ou desabilitar firewall temporariamente
Ping 192.168.100.x funciona, mas 172.16.x.x não	AllowedIPs incompleto	Verificar se incluiu a rede /24 do peer
Handshake ok, mas transfer 0 bytes	NAT ou roteamento	Verificar ip_forward e rotas
Chaves trocadas	Erro de copiar/colar	Regenerar e recopiar com cuidado

Comando útil de diagnóstico:

```
bash
# Verificar se kernel está encaminhando pacotes
sysctl net.ipv4.ip_forward
# Se retornar 0, habilite:
sudo sysctl -w net.ipv4.ip_forward=1
```

11. Desligamento e Limpeza

```
bash
# Em ambas as VMs
sudo wg-quick down wg0

# Remover alias de loopback (opcional)
sudo ip addr del 172.16.1.1/24 dev lo
sudo ip addr del 172.16.2.1/24 dev lo
```